

C# 프로그래밍

강의 12 — 종합 정리

Unity 게임 제작을 위한 C# 한 학기 요약

동명대학교 게임학부

강영민

2026년 1학기

이 강의의 목표

본 강의는 한 학기 동안 다룬 C# 언어의 핵심 개념을 **Unity 게임 제작이라는 관점에서** 다시 한번 통합적으로 정리한다. 단편적으로 알고 있던 문법, 클래스, 이벤트, 코루틴 등의 요소가 **왜 그렇게 생겼고, 게임 제작에 어떻게 쓰이는지**를 큰 그림으로 이해하는 것이 목표이다.

Contents

1 C#이란 무엇이고, Unity 게임 제작에 필요한 이유	3
1.1 C#의 정체성	3
1.2 왜 Unity는 C#을 선택했나?	3
1.3 이 학기에 우리가 배운 것	3
2 C#의 기본 문법 — 변수, 연산, 제어, 함수	5
2.1 변수와 기본 자료형	5
2.2 연산자	5
2.3 제어 구조	5
2.3.1 조건문 — if, switch	5
2.3.2 반복문 — for, while, foreach	6
2.4 함수(메서드)	6
3 객체지향 프로그래밍 기본 — 클래스	7
3.1 왜 클래스인가?	7
3.2 클래스 정의	7
3.3 인스턴스, 그리고 new	7
3.4 접근 제어자	7
3.5 프로퍼티 — 필드의 안전한 출입구	8
3.6 생성자 vs Unity의 Awake/Start	8
4 객체지향의 활용성 — 상속, 컴포넌트	9
4.1 상속(Inheritance) — “는 일종의” 관계	9
4.1.1 virtual / override / base	9
4.1.2 다형성(Polymorphism)	9
4.2 인터페이스(Interface) — “를 할 수 있는” 능력	9
4.3 컴포넌트(Component) — “를 가진다” 관계	10
4.3.1 상속 vs 컴포지션 — “is-a”와 “has-a”	10
5 유니티에서의 C# 활용 방식 — 프로그램 제어	11
5.1 MonoBehaviour 생명주기	11
5.2 SerializeField와 Inspector	11
5.3 이벤트 기반 프로그래밍 (event, Action)	11
5.4 코루틴(Coroutine) — “시간이 흐르는 동작”	12
5.5 메시지 시스템 정리	12
6 고급 테크닉 — 코드를 간결하게	13
6.1 식 본문 멤버 (=>)	13
6.2 람다식과 익명 함수	13
6.3 자동 구현 프로퍼티	13
6.4 문자열 보간 (\$"...")	13
6.5 Null 안전 연산자 (?., ??)	13
6.6 컬렉션/객체 초기화자	14
6.7 패턴 매칭 (switch 식)	14
6.8 using 디렉티브와 별칭	14
6.9 nameof와 디버깅	14

7	유니티에서 C#을 중심으로 게임 만들어 보기 — 수박게임	15
7.1	게임 컨셉	15
7.2	씬 구성	15
7.3	데이터 클래스 — FruitData	15
7.4	과일 컴포넌트 — Fruit	16
7.5	게임 매니저 — GameManager	17
7.6	입력 처리 — FruitSpawner	18
7.7	게임 오버 판정 — DeadLine	18
7.8	UI 연결 — HudBinder	19
7.9	Inspector 연결 절차 (요약)	19
7.10	이 게임에 들어 있는 한 학기 개념	19

1. C#이란 무엇이고, Unity 게임 제작에 필요한 이유

1.1 C#의 정체성

C#(C-Sharp)은 Microsoft가 2000년대 초에 발표한 객체지향 프로그래밍 언어이다. 표면적으로는 C와 Java의 문법을 닮았지만, 내부적으로는 .NET 런타임 위에서 동작하며 다음과 같은 현대 언어의 특성을 모두 갖춘다.

- 정적 타입(Static Typing) — 변수의 타입이 컴파일 시점에 결정. 대부분의 오류를 게임을 실행하기 전에 잡아낼 수 있다.
- 객체지향(Object-Oriented) — 데이터와 행동을 클래스라는 한 덩어리로 묶는다.
- 가비지 컬렉션(GC) — 메모리 해제를 자동으로 처리. C/C++의 delete를 잊을 일이 없다.
- 풍부한 표준 라이브러리 — 컬렉션, 파일 입출력, 비동기 처리, 네트워크 등이 기본 제공.
- 현대적 문법 — 람다, LINQ, 패턴 매칭, record, async/await 등을 갖춘다.

한 줄 요약

C#은 “C/Java의 안정감 + 스크립트 언어의 간결함”을 함께 노린 언어이다. 빠르게 동작하면서도 빠르게 작성할 수 있는 균형이 핵심이다.

1.2 왜 Unity는 C#을 선택했나?

Unity는 게임을 만들 때 C#을 “스크립팅 언어”로 사용한다. 엔진 핵심(렌더링, 물리, 메모리 관리)은 C++로 작성되어 있지만, 게임 로직(캐릭터의 행동, 점수 계산, UI 업데이트 등)은 거의 모두 C#으로 작성한다.

- 생산성 — GC, 정적 타입, 풍부한 라이브러리 덕분에 빠르게 작성하면서도 안정적이다.
- 성능 — IL(중간 코드)로 컴파일된 후 JIT/AOT로 네이티브에 가깝게 실행된다.
- 생태계 — 수많은 에셋, 튜토리얼, 라이브러리가 C# 기반.
- 컴포넌트와의 궁합 — 클래스 단위로 컴포넌트를 작성/부착하는 Unity 구조가 C#의 객체지향과 잘 맞는다.

Unity + C#의 사고 방식

Unity에서 C#으로 게임을 만든다는 것은 결국 다음과 같이 말한다.

1. 게임 오브젝트(GameObject)에
2. 컴포넌트(MonoBehaviour를 상속한 C# 클래스)를 부착하고
3. 컴포넌트의 메서드가 매 프레임/이벤트마다 호출되며 상태를 바꾸는 것이다.

1.3 이 학기에 우리가 배운 것

주제	핵심 내용
변수/제어/함수	데이터를 담고, 흐름을 갈래로 나누고, 재사용 가능한 동작을 정의

클래스	데이터(필드)와 동작(메서드)을 묶어 새 타입을 만든다
상속/인터페이스	공통점은 상위 타입으로 묶고, 다른 점은 override한다
이벤트	“일이 벌어졌을 때 알려달라” 는 느슨한 결합
코루틴	“시간이 흐르는 동안” 동작을 자연스럽게 표현
Unity 생명주기	Awake → Start → Update → ... → OnDestroy의 흐름

이번 강의는 위 모든 주제를 “**게임을 만드는 흐름**” 안에서 다시 한 번 재배치하여 정리한다.

2. C#의 기본 문법 — 변수, 연산, 제어, 함수

C#의 기본 문법은 C/Java를 한 번이라도 본 사람에게는 거의 같아 보인다. 다만 타입이 엄격하고 기본 자료형의 이름이 명확하다는 점이 두드러진다.

2.1 변수와 기본 자료형

값 형식(value type)과 참조 형식(reference type)이 명확히 구분된다.

범주	타입	설명	예
정수	int, long	32/64비트 정수	int hp = 100;
실수	float, double	부동소수점	float speed = 3.5f;
참/거짓	bool	true / false	bool isAlive = true;
문자/문자열	char, string	단일/연속 문자	string name = "Cube";
구조체	struct (값 형식)	좌표 등 가벼운 데이터	Vector3 p;
클래스	class (참조 형식)	일반적 객체	GameObject go;

```
int score = 0;           // 정수
float speed = 3.5f;      // 'f'는 float 리터럴 표시
bool isGameOver = false; // 참/거짓
string playerName = "P1"; // 문자열
var hp = 100;            // 'var'로 추론 (컴파일 타임에 int로 결정됨)
```

var는 “동적 타입”이 아니다

var는 컴파일러에게 타입 추론을 맡긴다는 의미일 뿐, 정해진 타입은 고정이다. var hp = 100; 이후에 hp = "백" 같은 코드는 컴파일 오류이다.

2.2 연산자

종류	기호
산술	+ - * / %
비교	== != < > <= >=
논리	&& !
할당/복합 할당	= += -= *= /=
증감	++ --
조건(삼항)	조건 ? A : B

```
int a = 7, b = 3;
int sum = a + b;           // 10
int rem = a % b;           // 1 (나머지)
bool inRange = (a > 0) && (a < 10);
string msg = inRange ? "범위 안" : "범위 밖";
```

2.3 제어 구조

2.3.1 조건문 — if, switch

```
if (hp <= 0) { Die(); }
else if (hp < 30) { ShowDanger(); }
else { /* 평소 동작 */ }

switch (state)
{
```

```

    case PlayerState.Idle:    Idle();    break;
    case PlayerState.Move:   Move();    break;
    case PlayerState.Attack: Attack();   break;
    default: break;
}

```

2.3.2 반복문 — for, while, foreach

```

for (int i = 0; i < enemies.Length; i++) { enemies[i].Move(); }

int frame = 0;
while (frame < 60) { frame++; }

foreach (var e in enemies) { e.Move(); } // 컬렉션을 “하나씩 꺼내며”

```

foreach는 “인덱스 없이 순회”에 쓴다

인덱스가 필요 없으면 foreach가 더 짧고 안전하다. 인덱스가 꼭 필요하다면 for를 쓴다. (예: 이전 원소와 비교)

2.4 함수(메서드)

C#의 함수는 클래스 안에 정의되는 메서드로 존재한다. 독립된 함수가 따로 있지 않다 — 모든 함수는 어떤 클래스에 속한다.

```

public class Calculator
{
    // 반환형 메서드명 (매개변수목록)
    public int Add(int a, int b)
    {
        return a + b;
    }

    public void Greet(string name = "World") // 기본 매개변수
    {
        Debug.Log($"Hello, {name}!");
    }
}

var c = new Calculator();
int r = c.Add(3, 5); // 8
c.Greet(); // "Hello, World!"
c.Greet("Unity"); // "Hello, Unity!"

```

메서드 시그니처의 4요소

1. 접근 제어자: public, private, protected, internal
2. 반환형: void는 “반환값 없음”
3. 메서드 이름: PascalCase가 관례
4. 매개변수 목록: 기본값, ref, out, params로 다양하게

3. 객체지향 프로그래밍 기본 — 클래스

3.1 왜 클래스인가?

게임 안에는 “플레이어”, “적”, “총알”, “HP 바” 같은 개체(*entity*)가 많다. 이런 개체는 모두

- 어떤 상태(데이터)를 가지고 (HP, 위치, 속도…)
- 어떤 행동(동작)을 한다 (이동, 공격, 죽음…)

이 둘을 따로 다루면 코드가 산만해진다. C#의 클래스는 상태와 행동을 한 덩어리로 묶어 새 타입을 만들어 주는 도구이다.

3.2 클래스 정의

```
public class Player
{
    // 1) 필드 (상태)
    private int hp = 100;
    private float speed = 3.5f;

    // 2) 프로퍼티 (필드에 접근하는 안전한 통로)
    public int Hp
    {
        get => hp;
        private set => hp = Mathf.Max(0, value);
    }

    // 3) 생성자 (객체가 만들어질 때 호출)
    public Player(int initialHp) { hp = initialHp; }

    // 4) 메서드 (행동)
    public void TakeDamage(int amount) { Hp -= amount; }
    public void Move(Vector3 dir) { /* 이동 처리 */ }
}
```

3.3 인스턴스, 그리고 new

클래스는 설계도이고, new로 실체(인스턴스, 객체)를 만든다.

```
Player p1 = new Player(100);
Player p2 = new Player(80);
p1.TakeDamage(20); // p1만 영향, p2는 그대로
```

3.4 접근 제어자

키워드	접근 범위
public	어디서든 접근 가능
private	같은 클래스 안에서만 접근 가능 (기본값)
protected	같은 클래스 + 상속받은 자식 클래스에서 접근 가능
internal	같은 어셈블리(같은 프로젝트) 안에서 접근 가능

“캡슐화”의 첫 단추

필드를 public으로 노출하지 마라. 대신 프로퍼티를 두라. 값이 “말도 안 되는 상태”가 되는 것을 막을 수 있다 — HP가 음수가 되거나, 속도가 무한이 되는 것을.

3.5 프로퍼티 — 필드의 안전한 출입구

```
private int hp;
public int Hp
{
    get { return hp; }
    set
    {
        hp = Mathf.Clamp(value, 0, 100); // 자동으로 0~100 사이로 보정
    }
}
```

이렇게 두면 `player.Hp = -50`을 시도해도 0으로 보정된다. 불변식(*invariant*)을 클래스 안에서 지킬 수 있다.

3.6 생성자 vs Unity의 Awake/Start

일반적인 C# 클래스는 `new`로 만들지만, `MonoBehaviour`는 `new`로 만들지 않는다. 대신 Unity가 `GameObject`에 부착하면서 자동으로 인스턴스화하고, 그 시점에 `Awake`, 그 다음에 `Start`를 호출한다.

Unity 컴포넌트에서는 생성자 대신 Awake/Start

- `Awake()` — 컴포넌트가 만들어진 직후, 자기 자신을 초기화할 때.
- `Start()` — 첫 `Update` 직전, 다른 컴포넌트와 연결한 뒤 초기화할 때.

4. 객체지향의 활용성 — 상속, 컴포넌트

4.1 상속(Inheritance) — “는 일종의” 관계

공통점은 부모(상위 클래스)로 묶고, 다른 점은 자식(하위 클래스)에서 override한다.

```
public class Enemy
{
    public int hp = 30;
    public virtual void Attack() { Debug.Log("기본 공격"); }
}

public class Slime : Enemy
{
    public override void Attack() { Debug.Log("끈적 공격!"); }
}

public class Dragon : Enemy
{
    public override void Attack()
    {
        base.Attack(); // 부모의 동작도 호출 가능
        Debug.Log("브레스 추가!");
    }
}
```

4.1.1 virtual / override / base

- virtual — “이 메서드는 자식이 재정의해도 좋다”
- override — “부모의 virtual 메서드를 자식 버전으로 갈아끼운다”
- base — 부모의 같은 메서드/생성자를 자식 안에서 직접 호출

4.1.2 다형성(Polymorphism)

부모 타입의 변수로 자식 객체를 다룰 수 있고, 실제 호출은 객체의 실제 타입을 따라간다.

```
Enemy[] enemies = new Enemy[] { new Slime(), new Dragon() };
foreach (var e in enemies) e.Attack();
// 출력: "끈적 공격!" 다음 "기본 공격" + "브레스 추가!"
```

4.2 인터페이스(Interface) — “를 할 수 있는” 능력

상속은 “는 일종의” 관계, 인터페이스는 “할 수 있는” 능력을 표현한다. 한 클래스는 여러 인터페이스를 동시에 가질 수 있다.

```
public interface IDamageable
{
    void TakeDamage(int amount);
}

public interface IInteractable
{
    void Interact();
}

public class Door : MonoBehaviour, IInteractable
{
    public void Interact() { /* 문 열기 */ }
}
```

```
public class Enemy : MonoBehaviour, IDamageable
{
    public void TakeDamage(int amount) { /* HP 감소 */ }
}
```

언제 상속, 언제 인터페이스?

공통 구현을 묶고 싶다면 상속. 공통 기능 약속만 묶고 싶다면 인터페이스. 의심스러우면 인터페이스가 더 유연하다.

4.3 컴포넌트(Component) — “를 가진다” 관계

Unity는 상속을 아주 얇게 쓰고, 대부분의 조합을 컴포넌트로 표현한다. 플레이어가 “이동할 수 있고 + 공격할 수 있고 + 인벤토리를 가진다”면 한 클래스에 모두 넣지 않고 세 개의 컴포넌트로 분리하여 GameObject에 함께 부착한다.

컴포넌트 패턴의 장점

- 각 컴포넌트는 단일 책임만 진다. (SRP)
- 같은 컴포넌트를 다른 *GameObject*에 재사용할 수 있다.
- 새로운 능력은 새 컴포넌트 추가로 끝난다. 기존 코드 수정이 적다.
- 디자이너가 *Inspector*에서 끼웠다 뺐다 하기 쉽다.

4.3.1 상속 vs 컴포지션 — “is-a”와 “has-a”

	상속 (is-a)	컴포지션 (has-a)
표현	<code>class Dragon : Enemy</code>	<code>GameObject + Health + Attacker</code>
유연성	한 부모만 가능, 깊어지면 어려워짐	능력을 자유롭게 조합
런타임 변경	어려움	<code>AddComponent/Destroy</code> 로 가능
Unity의 선호	얇게	기본 패러다임

5. 유니티에서의 C# 활용 방식 — 프로그램 제어

Unity에서 C#은 “언제 어느 시점에 호출될지”가 핵심이다. 일반 C# 프로그램은 Main()에서 시작해 내가 흐름을 직접 짠다. Unity는 반대로 엔진이 정해진 시점에 내 메서드를 호출한다 (inversion of control).

5.1 MonoBehaviour 생명주기

메서드	호출 시점
Awake	인스턴스가 만들어질 때. 자기 자신 초기화 (다른 컴포넌트 참조 전).
OnEnable	컴포넌트가 활성화될 때. 이벤트 구독하기 좋은 자리.
Start	첫 Update 직전. 다른 컴포넌트 참조 OK.
Update	매 프레임. 일반 게임 로직.
FixedUpdate	일정 간격(물리 스텝). Rigidbody 다룰 때.
LateUpdate	모든 Update 이후. 카메라 추적 등에 적합.
OnDisable	비활성화될 때. 이벤트 해제하기 좋은 자리.
OnDestroy	컴포넌트가 사라질 때. 최종 정리.

5.2 SerializeField와 Inspector

필드를 public으로 만들면 외부에서 마구 접근할 수 있다. 캡슐화는 지키면서 Inspector에는 노출하고 싶다면 [SerializeField].

```
public class Player : MonoBehaviour
{
    [SerializeField] private float speed = 3.5f; // Inspector에 보이지만 private 유지
    [SerializeField] private Transform target;
}
```

5.3 이벤트 기반 프로그래밍 (event, Action)

게임에서는 “언제 무슨 일이 벌어졌다”라는 사실을 알리고, 관심 있는 쪽이 알아서 반응하게 만드는 일이 잦다. 이때 event + Action을 쓰면 느슨한 결합이 된다.

```
public class ScoreManager : MonoBehaviour
{
    public event Action<int> OnScoreChanged; // "점수가 바뀜"을 알리는 신호

    private int score;
    public void Add(int v)
    {
        score += v;
        OnScoreChanged?.Invoke(score); // 구독자에게 알림
    }
}

public class ScoreUI : MonoBehaviour
{
    [SerializeField] private ScoreManager manager;
    void OnEnable() { manager.OnScoreChanged += HandleScore; }
    void OnDisable() { manager.OnScoreChanged -= HandleScore; }

    private void HandleScore(int s) { /* UI 갱신 */ }
}
```

이벤트의 ?.Invoke

OnScoreChanged?.Invoke(score);는 “구독자가 한 명도 없으면 그냥 넘어가라”는 안전장치이다. OnScoreChanged.Invoke(score);처럼 쓰면 구독자가 없을 때 NullReferenceException이 난다.

5.4 코루틴(Coroutine) — “시간이 흐르는 동작”

게임은 “1초 뒤에 시작”, “3초마다 적 등장”, “페이드 인 동안 천천히 알파 증가” 같은 시간이 흐르는 동안 진행되는 동작이 많다. 이런 일을 Update 안의 if (timer >= 1f) ... 식으로 풀면 코드가 복잡해진다. 코루틴은 “마치 그냥 절차를 적은 듯이” 표현하게 해준다.

```
IEnumerator FadeIn(SpriteRenderer sr, float dur)
{
    float t = 0f;
    while (t < dur)
    {
        t += Time.deltaTime;
        sr.color = new Color(1, 1, 1, t / dur);
        yield return null;           // 다음 프레임까지 대기
    }
    sr.color = Color.white;
}

void Start() { StartCoroutine(FadeIn(sprite, 1f)); }
```

yield return의 종류

- yield return null; — 다음 프레임까지 대기
- yield return new WaitForSeconds(1f); — 1초 대기 (게임 시간 기준)
- yield return new WaitForFixedUpdate(); — 다음 FixedUpdate까지
- yield break; — 코루틴 즉시 종료

5.5 메시지 시스템 정리

Unity의 “호출 흐름”을 정리하면 다음 세 갈래가 가장 중요하다.

1. **생명주기 메서드** — Awake/Start/Update/... : 엔진이 정해진 시점에 부른다.
2. **이벤트** — event Action : 내가 쏘면, 구독자들이 자동으로 반응.
3. **코루틴** — IEnumerator + yield : “시간”을 자연스럽게 다룬다.

이 세 가지를 알맞게 조합하는 것이 곧 Unity에서의 C# 프로그래밍이다.

6. 고급 테크닉 — 코드를 간결하게

C#은 같은 일을 여러 길로 쓸 수 있는 언어이다. 한 줄로 줄일 수 있는 곳을 알아두는 것만으로도 코드가 훨씬 깔끔해진다.

6.1 식 본문 멤버 (=>)

한 줄짜리 메서드/프로퍼티는 =>로 줄여 쓸 수 있다.

```
// 이전 방식
public int GetDouble(int x) { return x * 2; }
public int Square          { get { return value * value; } }

// 식 본문 멤버
public int GetDouble(int x) => x * 2;
public int Square          => value * value;
```

6.2 람다식과 익명 함수

이름 없는 함수를 값처럼 전달할 수 있다.

```
Action sayHi      = () => Debug.Log("Hi!");
Action<int> printHp = hp => Debug.Log($"HP={hp}");
Func<int,int,int> add = (a, b) => a + b;

button.onClick.AddListener(() => Debug.Log("클릭!"));
```

언제 람다, 언제 일반 메서드?

한 번만 쓰고 짧다면 람다. 여러 곳에서 쓰거나 복잡하다면 일반 메서드. 람다 안에 10줄짜리 로직을 넣지 마라.

6.3 자동 구현 프로퍼티

```
// 옛 방식
private int _score;
public int Score { get { return _score; } set { _score = value; } }

// 자동 구현
public int Score { get; set; }
public int Score { get; private set; } // 외부에선 읽기만
public int Score { get; init; }        // 생성 시점에만 쓰기 (C# 9 이상)
```

6.4 문자열 보간(\$"...")

```
// 옛 방식
Debug.Log("Score: " + score + ", HP: " + hp);
// 보간
Debug.Log($"Score: {score}, HP: {hp}");
Debug.Log($"속도={speed:F2}"); // 형식 지정도 가능
```

6.5 Null 안전 연산자(?., ??)

```
// ?. : 앞이 null이면 그냥 null 반환 (예외 없음)
int? len = name?.Length;

// ?? : 앞이 null이면 뒤의 값
string display = name ?? "Unknown";
```

```
// 결합
int safeLen = name?.Length ?? 0;

// 이벤트 호출에 자주 등장
OnScoreChanged?.Invoke(score);
```

6.6 컬렉션/객체 초기화자

```
var list = new List<int> { 1, 2, 3, 4 };

var p = new Player
{
    Name = "Hero",
    Hp = 100,
    Speed = 3.5f
};
```

6.7 패턴 매칭 (switch 식)

```
string Describe(int hp) => hp switch
{
    <= 0      => "사망",
    < 30      => "위험",
    < 80      => "양호",
    -        => "최상"
};
```

6.8 using 디렉티브와 별칭

```
using System.Collections.Generic;
using Random = UnityEngine.Random; // 별칭 (System.Random과 충돌 방지)
```

6.9 nameof와 디버깅

```
Debug.Log($"{nameof(player)}={player}"); // "player=..."
// 변수명을 문자열로 얻을 수 있어, 리팩토링 시에도 안전
```

“간결”과 “읽기 쉬움”은 다르다

=>, 람다, 삼항, 패턴 매칭은 읽기 쉬워질 때만 줄여라. 한 줄로 만들었는데 무슨 의미인지 1분 동안 봐야 한다면 일반 문장이 낫다.

7. 유니티에서 C#을 중심으로 게임 만들어 보기 — 수박게임

지금까지 배운 모든 요소를 하나의 미니 게임으로 묶어 본다. 대상 게임은 수박게임(*Suika Game*)이다. 물리, 충돌, 인스턴스화, 이벤트, 코루틴, UI가 모두 한 자리에 모이는 한 학기 종합 예제로 알맞다.

7.1 게임 컨셉

- 위에서 과일을 떨어뜨린다 (마우스 클릭).
- 같은 단계의 과일 두 개가 충돌하면 합쳐져 다음 단계가 된다.
- 단계가 올라갈수록 점수가 증가한다.
- 화면 위쪽 데드라인 위에 과일이 일정 시간 머물면 게임 오버.
- 가장 큰 단계(수박)에 도달하는 것이 목표.

이 게임 안에 들어 있는 한 학기 개념

클래스/필드/메서드, 캡슐화, 프로퍼티, event Action, 코루틴, Rigidbody2D + Collider2D 로 본 물리/충돌, Instantiate, Prefab, [SerializeField], 람다, =>, 문자열 보간.

7.2 씬 구성

Hierarchy

```
Scene
+-- Main Camera
+-- Walls          (3개 Edge/BoxCollider2D: 좌/우/바닥)
+-- DeadLine       (BoxCollider2D, 컴포넌트: DeadLine.cs)
+-- GameManager    (빈 GameObject, 컴포넌트: GameManager.cs)
+-- FruitSpawner   (빈 GameObject, 컴포넌트: FruitSpawner.cs)
+-- HUD (Canvas)
  +-- ScoreText (TMP_Text)
  +-- GameOverPanel (처음에는 비활성)
+-- FruitPrefab     (Assets에 보관: Rigidbody2D + CircleCollider2D
                    + SpriteRenderer + Fruit.cs)
```

7.3 데이터 클래스 — FruitData

단계별 반지름, 색, 합쳐졌을 때 얻은 점수를 한 곳에 모아 두는 정적 데이터 클래스.

```
// FruitData.cs --- 단계별 데이터 (정적)
using UnityEngine;

public static class FruitData
{
    // (반지름, 색, 합쳐졌을 때 얻는 점수)
    public static readonly (float radius, Color color, int points)[] Levels =
    {
        (0.30f, new Color(1.00f, 0.70f, 0.70f), 1), // 0: 체리
        (0.40f, new Color(1.00f, 0.55f, 0.55f), 3), // 1: 딸기
        (0.55f, new Color(1.00f, 0.75f, 0.50f), 6), // 2: 포도
        (0.70f, new Color(1.00f, 0.85f, 0.40f), 10), // 3: 귤
        (0.90f, new Color(1.00f, 0.55f, 0.20f), 15), // 4: 오렌지
        (1.10f, new Color(0.95f, 0.30f, 0.30f), 21), // 5: 사과
        (1.30f, new Color(0.95f, 0.85f, 0.60f), 28), // 6: 배
        (1.55f, new Color(1.00f, 0.95f, 0.30f), 36), // 7: 복숭아
    }
}
```



```

        (1.80f, new Color(1.00f, 0.90f, 0.10f), 45), // 8: 파인애플
        (2.10f, new Color(0.85f, 0.95f, 0.50f), 55), // 9: 멜론
        (2.50f, new Color(0.30f, 0.80f, 0.30f), 66), // 10: 수박
    };

    public const int Max = 10; // 마지막 단계 인덱스
}

```

튜플(tuple)을 활용한 데이터 묶기

(float radius, Color color, int points)처럼 **이름 붙은 튜플**을 쓰면 작은 데이터 묶음을 클래스를 새로 만들지 않고도 표현할 수 있다. `FruitData.Levels[3].radius`처럼 접근한다.

7.4 과일 컴포넌트 — Fruit

한 과일을 표현한다. `Rigidbody2D`와 `CircleCollider2D`로 물리에 참여하고, `OnCollisionEnter2D`에서 같은 단계의 과일과 부딪히면 정적 이벤트로 “이 둘을 합쳐 달라”고 알린다.

```

// Fruit.cs --- 과일 한 개
using System;
using System.Collections.Generic;
using UnityEngine;

[RequireComponent(typeof(Rigidbody2D), typeof(CircleCollider2D),
    typeof(SpriteRenderer))]
public class Fruit : MonoBehaviour
{
    // 두 같은 단계 과일이 부딪혔다는 신호 (정적 이벤트)
    public static event Action<Fruit, Fruit> OnMergePair;

    // 현재 살아 있는 모든 과일 (데드라인 판정에서 사용)
    public static readonly List<Fruit> All = new();

    public int Level { get; private set; }
    private bool merged; // 이미 합쳐졌으면 중복 처리 방지

    public void Setup(int level, Sprite circleSprite)
    {
        Level = level;
        var info = FruitData.Levels[level];
        transform.localScale = Vector3.one * (info.radius * 2f);
        var sr = GetComponent<SpriteRenderer>();
        sr.sprite = circleSprite;
        sr.color = info.color;
    }

    void OnEnable() => All.Add(this);
    void OnDisable() => All.Remove(this);

    void OnCollisionEnter2D(Collision2D col)
    {
        if (merged) return;
        var other = col.gameObject.GetComponent<Fruit>();
        if (other == null || other.merged) return;
        if (other.Level != Level) return;
        if (Level >= FruitData.Max) return;

        // 두 과일 중 하나만 이벤트를 쏘도록 ID로 한쪽을 정한다.
        if (GetInstanceID() < other.GetInstanceID())
        {
            merged = true; other.merged = true;
            OnMergePair?.Invoke(this, other);
        }
    }
}

```

“두 객체가 동시에 신호를 쏘는 문제”

같은 단계 두 과일이 부딪히면 OnCollisionEnter2D가 **양쪽 모두에서** 호출된다. 그대로 두면 합치기 이벤트가 두 번 발생한다. GetInstanceID()로 한쪽만 신호를 보내고, merged 플래그로 재진입 방지까지 한 것이 안전 장치이다.

7.5 게임 매니저 — GameManager

게임의 중심. 점수를 관리하고, 합치기 신호를 받아 새 과일을 만들고, 이벤트를 통해 UI/사운드 등에 변화를 알린다. Instance로 다른 컴포넌트가 접근하기 쉽도록 했다.

```
// GameManager.cs --- 점수, 합치기, 게임 오버
using System;
using UnityEngine;

public class GameManager : MonoBehaviour
{
    public static GameManager Instance { get; private set; }

    public event Action<int> OnScoreChanged;
    public event Action OnGameOver;

    [SerializeField] private Fruit fruitPrefab;
    [SerializeField] private Sprite circleSprite;

    public bool IsOver { get; private set; }
    public int Score { get; private set; }

    void Awake()
    {
        Instance = this;
        Fruit.OnMergePair += HandleMerge; // 합치기 신호 구독
    }

    void OnDestroy() => Fruit.OnMergePair -= HandleMerge;

    public Fruit Spawn(int level, Vector3 pos)
    {
        var f = Instantiate(fruitPrefab, pos, Quaternion.identity);
        f.Setup(level, circleSprite);
        return f;
    }

    private void HandleMerge(Fruit a, Fruit b)
    {
        if (IsOver) return;

        int newLevel = a.Level + 1;
        Vector3 mid = (a.transform.position + b.transform.position) * 0.5f;

        Destroy(a.gameObject);
        Destroy(b.gameObject);

        if (newLevel <= FruitData.Max) Spawn(newLevel, mid);

        Score += FruitData.Levels[newLevel - 1].points;
        OnScoreChanged?.Invoke(Score);
    }

    public void TriggerGameOver()
    {
        if (IsOver) return;
        IsOver = true;
        OnGameOver?.Invoke();
    }
}
```

7.6 입력 처리 — FruitSpawner

마우스를 클릭하면 현재 마우스 X 위치에서 과일을 떨어뜨린다. 난사 방지를 위해 쿨다운을 코루틴으로 관리한다.

먼저 확인할 것 — “Old Input” 사용을 위한 프로젝트 설정

이 예제는 `Input.GetMouseButtonDown(0)`, `Input.mousePosition`처럼 기존 입력 매니저 (Input Manager, 이른바 Old Input)의 API를 쓴다. 그런데 최근 Unity(2022 LTS 이후)는 프로젝트를 새로 만들 때 New Input System만 활성화되어 있는 경우가 많아서 `Input....` 호출이 컴파일 또는 실행 단계에서 오류로 막힌다.

이를 피하려면 Project Settings를 한 번 바꿔 줘야 한다.

1. Unity 에디터 메뉴에서 Edit → Project Settings... 를 연다.
2. 왼쪽 목록에서 Player를 선택한다.
3. 오른쪽의 Other Settings → Configuration 영역에 있는 Active Input Handling 항목을 찾는다.
4. 값을 Input System Package (New)에서 Input Manager (Old) 또는 Both로 변경한다.
5. 에디터가 재시작되며, 이후 `Input....` 호출이 정상 동작한다.

Both로 두면 두 시스템을 동시에 켜 두므로, 이 예제처럼 Old API를 쓰는 코드와 New Input System으로 작성한 다른 예셋이 같은 프로젝트에서 함께 동작한다.

```
// FruitSpawner.cs --- 마우스로 과일 드롭
using System.Collections;
using UnityEngine;

public class FruitSpawner : MonoBehaviour
{
    [SerializeField] private float dropY      = 4f;
    [SerializeField] private float xMin       = -3f;
    [SerializeField] private float xMax       = 3f;
    [SerializeField] private float cooldown    = 0.7f;
    [SerializeField] private int  maxStartLevel = 4; // 0~3단계까지만 떨어뜨림

    private bool canDrop = true;

    void Update()
    {
        if (GameManager.Instance.IsOver) return;
        if (!canDrop) return;
        if (!Input.GetMouseButtonDown(0)) return;

        Vector3 m = Camera.main.ScreenToWorldPoint(Input.mousePosition);
        float x = Mathf.Clamp(m.x, xMin, xMax);
        int level = Random.Range(0, maxStartLevel);

        GameManager.Instance.Spawn(level, new Vector3(x, dropY, 0f));
        StartCoroutine(Cooldown());
    }

    IEnumerator Cooldown()
    {
        canDrop = false;
        yield return new WaitForSeconds(cooldown);
        canDrop = true;
    }
}
```

7.7 게임 오버 판정 — Deadline

데드라인보다 위에 어느 과일이라도 머무는 시간을 누적하다가, 임계치를 넘으면 게임 오버.

```
// Deadline.cs --- 데드라인 위 누적 시간으로 게임 오버
using UnityEngine;
```

```

public class DeadLine : MonoBehaviour
{
    [SerializeField] private float warningDuration = 2f;

    private float aboveTime;

    void Update()
    {
        if (GameManager.Instance.IsOver) return;

        float lineY = transform.position.y;
        bool anyTop = false;

        foreach (var f in Fruit.All)
        {
            if (f.transform.position.y > lineY) { anyTop = true; break; }
        }

        aboveTime = anyTop ? aboveTime + Time.deltaTime : 0f;

        if (aboveTime >= warningDuration)
            GameManager.Instance.TriggerGameOver();
    }
}

```

7.8 UI 연결 — HudBinder

이벤트에 람다로 구독하여 점수와 게임 오버 표시를 갱신.

```

// HudBinder.cs --- 점수 / 게임 오버 UI
using UnityEngine;
using TMPro;

public class HudBinder : MonoBehaviour
{
    [SerializeField] private GameManager manager;
    [SerializeField] private TMP_Text scoreText;
    [SerializeField] private GameObject gameOverPanel;

    void OnEnable()
    {
        manager.OnScoreChanged += s => scoreText.text = $"Score: {s}";
        manager.OnGameOver += () => gameOverPanel.SetActive(true);
    }
}

```

7.9 Inspector 연결 절차 (요약)

1. **Project Settings → Player → Active Input Handling**을 Input Manager (Old) 또는 Both로 설정 (에디터 재시작). 그래야 FruitSpawner의 Input... 호출이 동작한다.
2. FruitPrefab: 빈 GameObject에 Rigidbody2D, CircleCollider2D, SpriteRenderer, Fruit.cs를 부착하고 Assets로 드래그하여 **Prefab**으로 저장.
3. GameManager의 fruitPrefab 슬롯에 위 Prefab을, circleSprite 슬롯에는 원형 스프라이트 한 장을 연결 (Unity 기본 UI/Skin/Knob 또는 직접 그린 원형 PNG).
4. HudBinder의 manager, scoreText, gameOverPanel 슬롯을 모두 **Inspector**에서 드래그하여 연결.
5. Walls는 좌/우/바닥 세 개의 Edge/BoxCollider2D로 구성.
6. DeadLine은 화면 상단을 가로지르도록 배치 (이 예제에서는 Update에서 y만 비교하므로 isTrigger 설정은 필요하지 않다).

7.10 이 게임에 들어 있는 한 학기 개념

개념	어디서 보였나
----	---------

변수/타입	int Level, bool merged, float aboveTime
연산/제어	if (other.Level != Level) return;, 삼항 anyTop ? ...: Of
함수/메서드	Setup, Spawn, HandleMerge, TriggerGameOver, Cooldown
클래스/필드/메서드	6개 클래스가 각자 역할을 명확히 나눠 가짐
정적 클래스/상수	FruitData, FruitData.Max
프로퍼티	Level { get; private set; }, IsOver, Score
캡슐화/접근제어	private set, [SerializeField] private
이벤트 (event Action)	Fruit.OnMergePair, OnScoreChanged, OnGameOver
정적 이벤트	Fruit.OnMergePair로 매니저 참조 없이 신호 송신
이벤트 구독/해제	Awake/OnDestroy, OnEnable/OnDisable
코루틴	FruitSpawner.Cooldown의 WaitForSeconds
람다	HudBinder의 += s => ... 구독
=> 식 본문	void OnEnable() => All.Add(this);
문자열 보간	\$"Score: {s}"
컴포넌트 패턴	Rigidbody2D + CircleCollider2D + SpriteRenderer + Fruit 조합
[RequireComponent]	Fruit가 자기에게 필요한 컴포넌트를 <i>Unity</i> 가 자동 보장
물리/충돌	Rigidbody2D.gravityScale, OnCollisionEnter2D
Prefab + Instantiate	Instantiate(fruitPrefab, pos, ...)
싱글톤(가벼운)	GameManager.Instance
컬렉션	List<Fruit> All, foreach로 순회

마무리

한 학기 동안 우리는 C# 문법 → 객체지향 → Unity의 컴포넌트/이벤트/코루틴 → 물리/충돌/Prefab으로 차근차근 올라왔다. 이 수박게임 한 편 안에 그 모든 것이 들어가 있다. 즉, 지금까지 배운 것만으로도 재미있는 게임을 처음부터 끝까지 만들 수 있다는 뜻이다. 남은 일은 많이 만들어 보는 것이다. 이 코드를 그대로 둔 채 과일 종류를 추가하거나, 콤보 효과음을 넣거나, Best Score를 저장해 보는 식의 작은 확장을 거듭하다 보면, 어느덧 자기만의 게임이 완성되어 있을 것이다.